

# PRÁCTICA ESPNOPW

Comunicación ESP-NOW entre ESP32-C6 usando MicroPython y uasyncio

*Sistemas Embebidos*

---

MicroPython · ESP-NOW

# 1. Objetivo

---

El objetivo de esta práctica es implementar un sistema de comunicación inalámbrica peer-to-peer entre dos microcontroladores ESP32-C6 utilizando el protocolo ESP-NOW programado en MicroPython. De manera complementaria, se integran los siguientes conceptos:

- Configuración del modo STA (Station Mode) del ESP32-C6.
- Obtención y uso de direcciones MAC para emparejamiento entre dispositivos.
- Comunicación bidireccional mediante el protocolo ESP-NOW.
- Programación concurrente usando uasyncio.
- Lectura no bloqueante de consola serial mediante uselect.
- Gestión de tareas asíncronas para transmisión y recepción simultánea de datos.
- Comunicación serial interactiva usando la terminal de Thonny IDE.

## 1.1 Objetivos Específicos

- Configurar el ESP32-C6 en modo STA para habilitar ESP-NOW.
- Obtener la dirección MAC de cada ESP32-C6 y utilizarla para registrar peers.
- Implementar transmisión de mensajes entre dos ESP32-C6 mediante ESP-NOW.
- Desarrollar tareas asíncronas con uasyncio para envío y recepción concurrente.
- Implementar lectura no bloqueante desde consola serial utilizando uselect.poll().
- Validar la comunicación inalámbrica en tiempo real desde la terminal de Thonny IDE.

## 1.2 Alcance de la Práctica

El alcance de esta práctica comprende exclusivamente la implementación del firmware en MicroPython utilizando ESP-NOW y programación asíncrona con uasyncio.

La práctica incluye:

- Obtención de direcciones MAC.
- Configuración inalámbrica.
- Emparejamiento de dispositivos.
- Envío y recepción de mensajes.
- Comunicación serial mediante consola.

## 2. Hardware Utilizado

---

### 2.1 Componentes

| Componente        | Cantidad | Descripción                                               |
|-------------------|----------|-----------------------------------------------------------|
| ESP32-C6          | 2        | Microcontroladores principales con soporte WiFi y ESP-NOW |
| Cable USB-C       | 2        | Programación y alimentación                               |
| PC con Thonny IDE | 2        | Entorno de desarrollo y consola serial                    |

## 3. Arquitectura del Software

---

La práctica está organizada en un archivo de micropython.

| Archivo   | Responsabilidad                        |
|-----------|----------------------------------------|
| ESPNOW.py | Lógica principal: código y bibliotecas |

## 4. Descripción Técnica Detallada

---

### 4.1 Obtención de la Dirección MAC

Antes de establecer comunicación ESP-NOW es necesario conocer la dirección MAC de cada ESP32-C6. Cada dispositivo posee una dirección única utilizada para el emparejamiento entre peers.

```
import network

wlan = network.WLAN(network.STA_IF)
wlan.active(True)

print(wlan.config('mac'))
```

Este código se utiliza para obtener la dirección MAC de un ESP32-C6 mediante la librería `network` de MicroPython. Primero, se importa el módulo `network`, el cual permite configurar y controlar las interfaces de red inalámbrica del microcontrolador.

Posteriormente, se crea un objeto `WLAN` utilizando `network.WLAN(network.STA_IF)`, donde `STA_IF` indica que el ESP32-C6 trabajará en modo estación (Station Mode). Este modo es necesario para utilizar funciones de comunicación inalámbrica como ESP-NOW. Después, la

instrucción `wlan.active(True)` activa físicamente el subsistema WiFi interno del microcontrolador. Finalmente, `wlan.config('mac')` obtiene la dirección MAC única asociada al ESP32-C6 y la imprime en consola. La dirección MAC funciona como un identificador físico del dispositivo dentro de la red y es indispensable en ESP-NOW, ya que cada ESP32 debe conocer la MAC del dispositivo remoto para poder registrarlo como peer y establecer comunicación inalámbrica directa.

## 4.2 Lógica Principal — ESPNOW.py

Este es el archivo central del proyecto. El mismo firmware se carga en ambos ESP32-C6. La única diferencia entre dispositivos es la dirección MAC configurada en la variable “mac”, que corresponde al peer remoto.

```
import network
import espnow
import uasyncio as asyncio
import uselect
import sys

mac = b'X\x8c\x811\x11\xbc'

wlan = network.WLAN(network.STA_IF)
wlan.active(True)

canal = espnow.ESPNow()
canal.active(True)

canal.add_peer(mac)

poll = uselect.poll()
poll.register(sys.stdin, uselect.POLLIN)

def enviar_mensaje(mensaje):
    canal.send(mac, mensaje.encode())

async def recibir_mensaje():
    while True:
        mac, mensaje = canal.recv(0)

        if mensaje:
            print("Recibido:", mensaje.decode())

        await asyncio.sleep_ms(50)

async def leer_consola():
    buffer_mensaje = ""

    while True:
        eventos = poll.poll(0)

        if eventos:
            caracter = sys.stdin.read(1)

            if caracter == '\n':
                if buffer_mensaje:
                    enviar_mensaje(buffer_mensaje)
                    buffer_mensaje = ""
```

```

        else:
            buffer_mensaje += character

        await asyncio.sleep_ms(20)

async def main():
    asyncio.create_task(recibir_mensaje())
    asyncio.create_task(leer_consola())

    while True:
        await asyncio.sleep(1)

asyncio.run(main())

```

### 4.2.1 Organización del código

| Responsabilidad |                                              |
|-----------------|----------------------------------------------|
| ESPNow          | Comunicación inalámbrica entre dispositivos. |
| uasyncio        | Ejecución concurrente de tareas              |
| uselect         | Lectura no bloqueante de consola             |
| sys.stdin       | Captura de caracteres desde Thonny           |
| main()          | Coordinación general del sistema             |

### 4.2.2 Configuración WiFi en Modo STA

ESP-NOW requiere que el hardware WiFi esté activo. Para ello se utiliza el modo STA:

```

wlan = network.WLAN(network.STA_IF)
wlan.active(True)

```

El modo STA permite:

- Activar radio WiFi.
- Utilizar ESP-NOW.
- Mantener compatibilidad con funciones inalámbricas del ESP32-C6.

Aunque ESP-NOW no usa Internet, sí utiliza el hardware físico del subsistema WiFi.

### 4.2.3 Inicialización de ESP-NOW

```
canal = espnow.ESPNow()  
canal.active(True)
```

La clase `ESPNow()` inicializa el protocolo inalámbrico de bajo consumo desarrollado por Espressif.

Características principales:

- Comunicación peer-to-peer.
- Baja latencia.
- No requiere router.
- Bajo consumo energético.
- Transmisión rápida de pequeños paquetes de datos.

### 4.2.4 Registro de Peers

```
canal.add_peer(mac)
```

Registra la dirección MAC del dispositivo remoto autorizado para intercambiar datos.

ESP-NOW funciona mediante emparejamiento explícito:

- Cada dispositivo debe conocer la MAC del otro.
- Solo los peers registrados pueden intercambiar mensajes.

### 4.2.5 Lectura No Bloqueante con `uselect`

La lectura de la terminal serial normalmente es una operación bloqueante, es decir, el programa se detiene mientras espera que el usuario escriba información (input). Para evitar este comportamiento y permitir que otras tareas continúen ejecutándose simultáneamente, se utiliza la librería `uselect`.

Primero se crea un objeto `poll` mediante `uselect.poll()`, el cual funciona como un monitor de eventos de entrada y salida. Posteriormente, con `poll.register(sys.stdin, uselect.POLLIN)` se registra la entrada estándar de la consola (`stdin`) para detectar eventos de lectura disponibles (`POLLIN`).

```
poll = uselect.poll()  
poll.register(sys.stdin, uselect.POLLIN)
```

Esto permite:

- Detectar si existe información disponible en consola.
- Continuar ejecutando otras tareas mientras no haya entrada.

## 4.2.6 Envío de Mensajes

```
def enviar_mensaje(mensaje):  
    canal.send(mac, mensaje.encode())
```

Proceso de transmisión:

1. El usuario escribe texto en Thonny.
2. El string se convierte a bytes usando `.encode()`.
3. ESP-NOW transmite el paquete inalámbricamente al peer remoto.

## 4.2.7 Recepción de mensajes

La función `recibir_mensaje()` está definida utilizando *async def*, lo que indica que se trata de una función asíncrona de `uasyncio`. Esto permite que la tarea pueda ejecutarse concurrentemente junto con otras funciones asíncronas sin bloquear el procesador.

```
async def recibir_mensaje():  
    while True:  
        mac, mensaje = canal.recv(0)  
  
        if mensaje:  
            print("Recibido:", mensaje.decode())  
  
        await asyncio.sleep_ms(50)
```

La instrucción `canal.recv(0)` realiza una lectura del canal inalámbrico y retorna dos valores:

- la dirección MAC del dispositivo que envió el mensaje (`mac`).
- el contenido del mensaje recibido (`mensaje`).

La dirección MAC permite identificar qué peer transmitió la información, mientras que `mensaje` contiene los datos recibidos en formato bytes. Si existe un mensaje válido, éste se convierte nuevamente a texto utilizando `mensaje.decode()` y posteriormente se imprime en consola mediante `print("Recibido:", mensaje.decode())`.

Finalmente, la instrucción `await asyncio.sleep_ms(50)` suspende temporalmente la tarea durante 50 milisegundos y cede tiempo de ejecución a otra tarea asíncrona de `uasyncio`.

## 4.2.8 Lectura en terminal

La función `leer_consola()` está definida utilizando `async def`, lo que indica que se trata de una tarea asíncrona administrada por `uasyncio`. Su objetivo es leer continuamente los caracteres escritos por el usuario en la terminal serial de Thonny sin bloquear la ejecución del resto del sistema.

```
async def leer_consola():
    buffer_mensaje = ""

    while True:
        eventos = poll.poll(0)

        if eventos:
            caracter = sys.stdin.read(1)

            if caracter == '\n':
                if buffer_mensaje:
                    enviar_mensaje(buffer_mensaje)
                    buffer_mensaje = ""
            else:
                buffer_mensaje += caracter

        await asyncio.sleep_ms(20)
```

Inicialmente se crea la variable `buffer_mensaje`, utilizada para almacenar temporalmente los caracteres introducidos desde la consola hasta formar un mensaje completo

Dentro del bucle infinito `while True`, la variable `eventos` almacena el resultado de la instrucción `poll.poll(0)`, la cual hace uso del método `poll()` perteneciente al objeto `poll`, previamente creado mediante `uselect.poll()`. Este método se encarga de revisar si existen eventos de entrada o salida registrados en el sistema, en este caso la consola serial (`sys.stdin`). El parámetro 0 indica un timeout inmediato, por lo que el método retorna instantáneamente sin detener la ejecución del programa mientras espera datos.

Si se detecta actividad en la consola, se lee un carácter individual mediante `sys.stdin.read(1)`. Cada carácter recibido se agrega al buffer utilizando `buffer_mensaje += caracter`.

Cuando el usuario presiona ENTER (`\n`), el programa interpreta que el mensaje está completo. Se llama a la función `enviar_mensaje(buffer_mensaje)` para transmitir el texto mediante ESP-NOW y posteriormente el buffer se limpia para preparar la recepción de un nuevo mensaje. Finalmente, la instrucción `await asyncio.sleep_ms(20)` pausa temporalmente la tarea durante 20 milisegundos y cede tiempo de ejecución

### 4.2.9 Creación de tareas asíncronas en main

La función `main()` está definida utilizando `async def`, lo que indica que se trata de una función asíncrona administrada por `uasyncio`. Su propósito es crear y mantener activas las tareas concurrentes del sistema.

```
async def main():
    asyncio.create_task(recibir_mensaje())
    asyncio.create_task(leer_consola())

    while True:
        await asyncio.sleep(1)
```

Dentro de la función se utilizan las instrucciones `asyncio.create_task(recibir_mensaje())` y `asyncio.create_task(leer_consola())`, las cuales crean y registran dos tareas asíncronas independientes dentro del event loop de `uasyncio`. Gracias a `create_task()`, ambas funciones pueden ejecutarse concurrentemente de manera cooperativa sin bloquear el procesador.

Posteriormente, se implementa un bucle infinito `while True` cuya función principal es mantener viva la ejecución del programa y evitar que la función `main()` finalice. Dentro de este bucle se utiliza `await asyncio.sleep(1)`, que suspende temporalmente la tarea principal durante un segundo y cede tiempo de ejecución a `uasyncio`, permitiendo que las demás tareas continúen ejecutándose normalmente. Sin este bucle, la función `main()` terminaría inmediatamente después de crear las tareas y el event loop finalizaría, deteniendo todo el sistema asíncrono.

### 4.2.10 Creación de tareas asíncronas en main

```
asyncio.run(main())
```

La instrucción `asyncio.run(main())` se encarga de iniciar el sistema asíncrono administrado por `uasyncio`. El método `run()` recibe como argumento la función asíncrona `main()`, la cual actúa como punto de entrada principal del programa. Internamente, `asyncio.run()` crea y ejecuta el event loop o scheduler cooperativo de `uasyncio`, responsable de administrar todas las tareas asíncronas registradas mediante `create_task()`. Una vez iniciado el scheduler, la función `main()` comienza su ejecución y, a partir de ella, se lanzan concurrentemente las tareas `recibir_mensaje()` y `leer_consola()`. Gracias a este mecanismo, el ESP32-C6 puede ejecutar múltiples tareas aparentemente al mismo tiempo sin utilizar multithreading real, alternando cooperativamente el tiempo de CPU entre cada coroutine mediante las instrucciones `await`. Sin `asyncio.run(main())`, ninguna de las funciones asíncronas del sistema comenzaría a ejecutarse.

## 5. Flujo General del Sistema

El siguiente diagrama describe el flujo de ejecución completo del firmware desde el arranque:

```
INICIO → Importar módulos → activar WiFi STA → inicializar ESP-NOW
      ↓
Registrar peer mediante MAC → crear objeto poll() → registrar sys.stdin
      ↓
asyncio.run(main())
      ↓
SCHEDULER UASYNCIO
    ├── crear tarea recibir_mensaje()
    └── crear tarea leer_consola()

BUCLE ASÍNCRONO CONCURRENTES

recibir_mensaje():
  ├── canal.recv(0)
  ├── [mensaje recibido?]
  │   └── decodificar mensaje → imprimir en consola
  └── await asyncio.sleep_ms(50)

leer_consola():
  ├── poll.poll(0)
  ├── [evento en consola?]
  │   ├── leer carácter desde stdin
  │   │   ├── [ENTER presionado?]
  │   │   │   └── enviar_mensaje(buffer)
  │   └── almacenar carácter en buffer
  └── await asyncio.sleep_ms(20)
```

## 6. ESPNOW/uasyncio — Conceptos Clave

ESP-NOW es un protocolo de comunicación inalámbrica desarrollado por Espressif basado en tramas IEEE 802.11. Permite establecer comunicación directa entre dispositivos ESP32 sin necesidad de un router WiFi o conexión a Internet. Su principal ventaja es la baja latencia y el reducido consumo energético, características especialmente útiles en sistemas embebidos y aplicaciones IoT.

En esta práctica:

- El ESP32-C6 utiliza el modo STA (Station Mode) para habilitar el hardware WiFi necesario para ESP-NOW.
- Ambos ESP32-C6 funcionan simultáneamente como transmisores y receptores (peers simétricos).
- Cada dispositivo debe registrar la dirección MAC del peer remoto mediante `add_peer(mac)` para permitir la comunicación.
- La transmisión de datos se realiza inalámbricamente mediante paquetes enviados con `canal.send()` y recibidos mediante `canal.recv()`.

Cada transmisión ESP-NOW sigue una lógica simplificada:

*[MAC destino] → [Datos] → transmisión inalámbrica → recepción en peer remoto*

Por otra parte, uasyncio es una implementación ligera de programación asíncrona para MicroPython basada en ejecución cooperativa de tareas (coroutines). Permite ejecutar múltiples procesos concurrentemente sin utilizar multithreading real ni múltiples núcleos de procesamiento. En esta práctica:

- recibir\_mensaje() monitorea continuamente la recepción de mensajes ESP-NOW.
- leer\_consola() supervisa la entrada de texto desde la terminal serial de Thonny.
- asyncio.create\_task() registra ambas tareas dentro del scheduler cooperativo.
- await asyncio.sleep\_ms() cede tiempo de CPU para permitir la ejecución concurrente de las demás tareas.

Gracias a esta arquitectura, el ESP32-C6 puede enviar y recibir mensajes inalámbricos al mismo tiempo que monitorea la consola serial, manteniendo el sistema responsivo y evitando bloqueos durante la ejecución.

## 7. Dependencias y Bibliotecas

| Módulo/Biblioteca | Origen             | Uso en el proyecto                                         |
|-------------------|--------------------|------------------------------------------------------------|
| network           | MicroPython stdlib | Configuración y activación de la interfaz WiFi STA         |
| espnow            | MicroPython stdlib | Comunicación inalámbrica peer-to-peer mediante ESP-NOW     |
| uasyncio          | MicroPython stdlib | Programación asíncrona y ejecución concurrente de tareas   |
| uselect           | MicroPython stdlib | Lectura no bloqueante de la consola serial                 |
| sys               | MicroPython stdlib | Acceso a stdin para captura de caracteres desde Thonny IDE |

## 8. Conclusiones

---

Esta práctica demuestra la implementación exitosa de comunicación inalámbrica peer-to-peer entre dos ESP32-C6 utilizando ESP-NOW y programación concurrente en MicroPython.

Principales resultados:

- ESP-NOW permite comunicación rápida y eficiente sin infraestructura WiFi externa.
- uasyncio facilita la ejecución concurrente de múltiples tareas sin multithreading real.
- La lectura no bloqueante mediante uselect mantiene el sistema responsivo.
- La arquitectura basada en tareas asíncronas mejora la escalabilidad del firmware.
- El uso de Thonny IDE simplifica las pruebas interactivas y depuración serial.
- ESP-NOW representa una alternativa eficiente para aplicaciones embebidas de baja latencia y bajo consumo energético.

---

*Práctica ESPNOW*

## 8. Referencias

- MicroPython. (s.f.). espnow — support for the ESP-NOW wireless protocol. Recuperado de: [MicroPython ESP-NOW Documentation](#)
- MicroPython. (s.f.). uasyncio — asynchronous I/O scheduler. Recuperado de: [MicroPython uasyncio Documentation](#)
- MicroPython. (s.f.). network.WLAN — control built-in WiFi interfaces. Recuperado de: [MicroPython WLAN Documentation](#)
- Espressif Systems. (s.f.). ESP-NOW Wireless Communication Protocol. Recuperado de: [ESP-IDF ESP-NOW Documentation](#)
- Thonny Team. (s.f.). Thonny Python IDE Documentation. Recuperado de: [Thonny Documentation](#)

## 8. Anexos

```
# Importa el módulo network para habilitar y configurar la interfaz WiFi
import network

# Importa la librería ESP-NOW para comunicación inalámbrica peer-to-peer
import espnow

# Importa uasyncio y le asigna el alias asyncio para programación asíncrona
import uasyncio as asyncio

# Importa uselect para realizar lectura no bloqueante de la consola
import uselect

# Importa sys para acceder a la entrada estándar (stdin) de Thonny
import sys

# Dirección MAC del ESP32-C6 remoto con el que se intercambiarán mensajes
mac = b'X\x8c\x81\x11\xbc'

# Crea un objeto WLAN en modo estación (STA)
# ESP-NOW requiere que el hardware WiFi esté activo
wlan = network.WLAN(network.STA_IF)

# Activa la interfaz WiFi del ESP32-C6
wlan.active(True)

# Crea un objeto ESPNow para gestionar la comunicación ESP-NOW
canal = espnow.ESPNow()

# Activa el protocolo ESP-NOW
canal.active(True)

# Registra la MAC remota como peer autorizado
# Solo los peers registrados pueden intercambiar mensajes
canal.add_peer(mac)

# Crea un objeto poll para monitorear eventos de entrada/salida
poll = uselect.poll()

# Registra la consola serial (stdin) para detectar datos disponibles
# POLLIN indica que se monitorearán eventos de lectura
poll.register(sys.stdin, uselect.POLLIN)

# Función encargada de transmitir mensajes mediante ESP-NOW
def enviar_mensaje(mensaje):

    # Convierte el texto a bytes mediante encode()
    # y lo envía al peer remoto identificado por la MAC
    canal.send(mac, mensaje.encode())

# Tarea asíncrona encargada de recibir mensajes ESP-NOW
async def recibir_mensaje():

    # Bucle infinito para monitorear continuamente el canal inalámbrico
    while True:

        # Lee el canal ESP-NOW sin bloquear la ejecución
        # Retorna:
        #   mac -> dirección MAC del remitente
```

```

# mensaje -> datos recibidos en formato bytes
mac, mensaje = canal.recv(0)

# Verifica que efectivamente se haya recibido un mensaje
if mensaje:

    # Convierte los bytes a texto y los muestra en consola
    print("Recibido:", mensaje.decode())

# Pausa cooperativa de 50 ms
# Permite que otras tareas asincronas utilicen el procesador
await asyncio.sleep_ms(50)

# Tarea asíncrona encargada de leer texto desde la consola de Thonny
async def leer_consola():

    # Buffer temporal donde se almacenan los caracteres escritos
    buffer_mensaje = ""

    # Bucle infinito de monitoreo de la consola
    while True:

        # Consulta si existen eventos registrados en stdin
        # El parámetro 0 indica retorno inmediato (sin bloqueo)
        eventos = poll.poll(0)

        # Si existe actividad en la consola
        if eventos:

            # Lee un único carácter desde stdin
            caracter = sys.stdin.read(1)

            # Si el usuario presionó ENTER
            if caracter == '\n':

                # Verifica que el buffer no esté vacío
                if buffer_mensaje:

                    # Envía el mensaje completo mediante ESP-NOW
                    enviar_mensaje(buffer_mensaje)

                    # Limpia el buffer para comenzar un nuevo mensaje
                    buffer_mensaje = ""

            else:

                # Agrega el carácter recibido al buffer
                buffer_mensaje += caracter

        # Pausa cooperativa de 20 ms
        # Cede tiempo de ejecución a otras tareas asincronas
        await asyncio.sleep_ms(20)

# Función principal del sistema asíncrono
async def main():

    # Crea una tarea para recepción de mensajes
    asyncio.create_task(recibir_mensaje())

    # Crea una tarea para lectura de consola
    asyncio.create_task(leer_consola())

    # Bucle infinito para mantener viva la función principal
    while True:

        # Pausa cooperativa de 1 segundo
        # Evita que main() termine y finalice el event loop
        await asyncio.sleep(1)

```

```
# Inicia el scheduler (event loop) de uasyncio
# main() actúa como punto de entrada principal del programa
asyncio.run(main())
```